QA Guide — How to Write API Test Cases

How to Write API Test Cases

Version: 1.0 (English companion to Vietnamese v3.0) · **For:** QA Interns and Junior QA · **Goal:** Learn to prepare test data, send an API request in Postman, write a clear test case, and report a bug that a developer can repeat.

Vietnamese version: Hướng dẫn viết API Test Case — Tiếng Việt.

> **Start here:** Read sections 1–5 first. You do not need to know every technical word. Start with one good test case, then write more cases. Some URLs and JSON examples below are for learning only. They are **not confirmed live API requests**.

1. What is an API test case?

An **API** lets two systems send information to each other. Example: a partner enters hotel details and clicks Submit. The website sends the form data to an API. The API checks the data and creates an application or returns an error.

A **test case** explains one thing to check: **What do I send? What should happen? What really happened?**

Word	Simple meaning
Request	Information you send to an API.
Response	Information the API sends back.
URL / Endpoint	The address of an API operation.
Method	The action: GET reads, POST submits/creates or runs an action, PATCH changes part of a record, PUT replaces data as defined by the API, DELETE removes data.
Headers	Extra request information, such as JSON type or login token.
Payload / Body	The data you send, often in JSON format.
HTTP status	A result number. For example, 2xx often means success, 4xx often

	means a request problem or no permission, and 5xx often means a server problem. Always check the API contract.
Validation	Checking whether input data follows the rules.
Test data / Fixture	Data prepared for testing.
Expected / Actual	What should happen / What you saw when you ran the test.

2. What do you need before testing?

Get the information below from Jira, Confluence, the API document (OpenAPI), and your QA Lead or developer. **Do not guess.**

Item	Where to get it	Example
Requirement / Jira	Your Jira ticket and its Acceptance Criteria (rules for accepting the feature).	OP-1593 — Registration V2 QA
API document / OpenAPI	The document for the API version running in your test environment.	STAGING OpenAPI link: **[QA fills in]**
Base URL	The DEV or STAGING environment.	`{{baseUrl}}` = **[real URL]**
Method and endpoint	The current OpenAPI document.	`POST {{baseUrl}} {{registrationPath}}`
Login / token	QA Lead or environment owner.	Public API or Bearer token? **[check]**
Test data	Safe QA data and real reference values from STAGING.	Country code, location code, valid uploaded file IDs.

Build version	The deployed FE, Gateway and service version.	Commit SHA or release ID: **[QA fills in]**

> ⚠ **If the API document or expected result is missing:** write **BLOCKED — API contract missing** and ask your QA Lead or developer. Do not create your own URL, field names, error codes, or rules.

3. How to send a request in Postman

1. Open Postman and create an **HTTP Request**.
2. Select the correct **Method**, such as POST. Check OpenAPI first.
3. Enter the **URL**, such as `{{baseUrl}}{{registrationPath}}`, with real variable values.
4. Open **Headers**. Add `Content-Type: application/json` if the API accepts JSON. Add a token or other headers only if the API needs them.
5. Open **Body → raw → JSON**. Paste the full request data from your approved test fixture. If the API expects file upload or multipart data, use the type stated in OpenAPI.
6. Click **Send**. Write down the HTTP status, response body, error code (if any), and request ID (if available).
7. Check the **data after the request**. After success, use an allowed GET API to read the saved record. After a failed request, check that no wrong record was created or changed.
8. Write your **Actual Result**. Add a safe screenshot or response record as evidence.

Example URL, not real: `https://api.example.test/v2/partner-registrations`. Base URL: `https://api.example.test`. Endpoint path: `/v2/partner-registrations`. **Never use this example URL for STAGING tests.**

4. What must every API test case include?

Copy this table when you write a test case. **One test case should check one clear result.**

Field	What to write
Test Case ID	Example: `REGV2-API-002`.
Title	Example: “Reject registration when phone country is missing.”
Requirement	Jira link and the rule you test.
Priority / Test layer	Priority and API / FE / Integration.

Environment	DEV or STAGING, test date, and deployed version/SHA.
Preconditions	What must be ready first: login/role, test record, location, valid files.
Method + URL	The exact method and real API URL from OpenAPI.
Headers / Auth / Parameters	Required headers, login, path/query values, and retry key if supported.
Payload / Test data	Full JSON or multipart data. Show what you change from valid data.
Steps	Simple numbered steps another QA can follow.
Expected response	HTTP status, response fields, error code, field error; all from the real contract.
Expected data result	Was a record created or changed? How will you check?
Actual result	What you really saw after running the test.
Status and evidence	PASS / FAIL / NOT VERIFIED / BLOCKED; request ID, response, screenshot, bug link.

5. Full example: Missing phoneCountry in Accommodation Registration V2

Requirement: OP-1589 and OP-1505. The user must select a phone country, even when the phone number already has an international prefix, such as `+84`.

Test ID / Title	`REGV2-API-002` — Reject registration without `primaryContact.phoneCountry`.

Before testing	Prepare a **fully valid** V2 registration request: correct property/contact details, valid location codes, accepted uploaded files, and all required acknowledgements (confirm each field against the real OpenAPI). Use fake QA data, not personal data.
Method and URL	`POST {{baseUrl}} {{registrationPath}}` — method and path must be checked against STAGING OpenAPI. This page does **not** confirm the real endpoint.
Headers	`Content-Type: application/json` if accepted; add other headers only if OpenAPI requires them.
Test data change	Start with your fully valid request. **Remove only** `primaryContact.phoneCountry`. Keep `primaryContact.phone` in valid international E.164 format.
Steps	1. Check the original data is valid. 2. Remove phoneCountry only. 3. Send the POST request. 4. Save the response. 5. Check that no new Partner Request was created.
Expected response	The API rejects the request and returns an error for the phone-country field. **Exact HTTP status, error code, message key and field error: [QA fills in from real OpenAPI/error registry]**.

Expected data	No Partner Request is created by this invalid request.
Actual and status	**[Fill in after testing]**. Before testing, use `NOT VERIFIED`, not PASS.

5.1 Small JSON example — for learning only

```
{
  "propertyName": "QA Sample Hotel",
  "countryCode": "VN",
  "administrativeAreaCode": "<VALID_LOCATION_CODE>",
  "primaryContact": {
    "fullName": "QA Tester",
    "email": "qa-sample@example.test",
    "phone": "+84900000000"
  }
}
```

This example has no `phoneCountry`. It also does **not** include all V2 fields, documents, photos, or acknowledgements. **Do not send it as a real V2 request.** For a real test, use the complete valid V2 JSON and remove **only** phoneCountry. If many fields are missing, you may receive an error for a different field.

5.2 How to record test results

```
{
  "testCaseId": "REGV2-API-002",
  "environment": "STAGING",
  "method": "POST",
  "url": "{{baseUrl}}{{registrationPath}}",
  "testDataFile": "valid-v2-minus-phoneCountry.json",
  "expectedHttpStatus": "<CHECK_OPENAPI>",
  "expectedErrorCode": "<CHECK_ERROR_REGISTRY>",
  "actualHttpStatus": "NOT_RUN",
  "actualErrorCode": "NOT_RUN",
  "requestId": "NOT_RUN",
  "dataCheck": "No Partner Request created",
  "status": "NOT VERIFIED",
  "evidenceLink": "<ADD_AFTER_TEST>"
}
```

6. How to prepare good test data

Start with **one full, valid request** (a *valid fixture*). For a test that checks one error, change **one thing only**. This helps you understand why the API returned an error.

Test	Data change	What to check
Valid request	Use the complete approved valid data.	One application is saved; the receipt and saved data are correct.
Missing phone country	Remove phoneCountry; keep a	The API does not guess the phone

	valid phone number.	country from the number.
Wrong location	Use an area code that does not belong to the selected country.	The API rejects the wrong country/area combination.
No business document	Use zero documents.	The required document rule is enforced.
Not enough photos	Use only one photo.	The required 2–5 photo rule is enforced.
Send the same request again	Use the same request data and same retry/idempotency key, if supported.	The system does not create two Partner Requests.
Wrong user permission	Use a test account without the required access for a protected API.	The user cannot read or change data they do not own.

For Accommodation V2, OP-1590 records these technical limits: **1–5 business documents, 2–5 photos, up to 5 MiB per file, 20 MiB total**. Document types: PDF/JPEG/PNG. Photo types: JPEG/PNG. Confirm the current deployed OpenAPI/Media rules before using these as exact runtime expected results.

Useful test data file names: `valid-registration-v2.json`, `missing-phone-country.json`, `invalid-location-child.json`. Write down what changed from the valid file.

7. Common types of API tests

Type	Simple question	Example
Positive	Does the API work with correct data?	Full valid form → one application.
Negative	Does the API reject wrong data?	Wrong email format → clear error.
Boundary	What happens at and beyond a limit?	Two photos allowed, one photo rejected.

Permission	Can a person without access use this API?	Reviewer cannot open another user's protected record.
Duplicate / Retry	Will sending a request again create duplicates?	Repeat the same request after a timeout.
Integration	Do different services handle the same data correctly?	FE → Gateway → Partner → Admin readback.
Regression	Did the new change break an old feature?	V2 does not break supported V1 registration.

You do not need every test type for every API. Choose the types that matter for the requirement.

8. How to check an API response

If the API succeeds: do not check only the HTTP status. Check the response fields and the new application ID. Then use an allowed GET API to confirm that the data was saved correctly, including documents and photos.

If the API returns an error: check the HTTP status and the error fields stated in the API document, such as `errorCode`, `messageKey`, or `fieldErrors`. Confirm no wrong data was created or changed.

If the request times out: do not say “failed” right away. The server may have saved the record even if Postman did not get the response. Use the approved way to check status or retry. Ask the API owner if the correct action is not documented.

Helpful words: *Idempotency* = sending the same operation again does not create the same effect twice. *Persistence* = data stays saved. *Readback* = reading the saved data again. *Contract gap* = the document does not say what the expected result should be. *Fail closed* = the system rejects an action when required proof or access is missing.

9. How to record results and report bugs

Status	When to use it
PASS	You ran the test. The response and saved data match the documented expected result.

FAIL	You ran the test. You have evidence that the actual result is different from the documented expected result.
NOT VERIFIED	The test has not run, or you have not checked enough (for example, saved data was not checked).
BLOCKED	You cannot run or judge the test because the API document, access, data, or test environment is missing.

Simple bug report: Bug title → Environment/build → Test Case ID and Jira → Steps → Method/URL and safe request data → Expected → Actual HTTP/body → Request ID → screenshot/log. Ask which team owns the issue; a Gateway error response does not always mean the Gateway caused the problem.

⚠ **Keep test data safe.** Do not save passwords, access tokens, private signing links, real customer contact information, or real legal documents in Confluence or Jira. Use fake QA data. Hide Authorization headers and personal details in screenshots.

10. Practice task for QA Interns

Using OP-1593, write **at least five test cases** for Accommodation Registration V2: (1) valid request, (2) missing phoneCountry, (3) wrong country/area match, (4) missing documents or photos, (5) repeated request and duplicate safety.

Every test case must have: **Method, real URL, headers/auth, full request payload, steps, expected response, expected saved data, actual response, status, and evidence**. If an API detail is missing, mark it BLOCKED and say who can provide it.

11. QA Lead checklist

- Does the test case link to a real requirement and current OpenAPI?
- Are the method, URL, headers and body real, not copied from the example URL in this guide?
- Does the valid test data have all required fields, files and acknowledgements?
- Are expected HTTP statuses and error codes based on the official API contract?
- Can QA check whether data was saved or changed?
- Can another QA run the test using the written steps?
- Are secrets and personal data hidden?

12. Official references

API Standard · OpenAPI Governance, Error Contract & Docs Portal · GNS-FORM-001 — Global Form UX & Validation

Accommodation V2: OP-1505 (V2 policy) · OP-1589 (fields and validation) · OP-1590 (files) · OP-1591 (API errors) · OP-1592 (saved data and retries) · OP-1593 (QA tests) · REG-01–05 Figma.